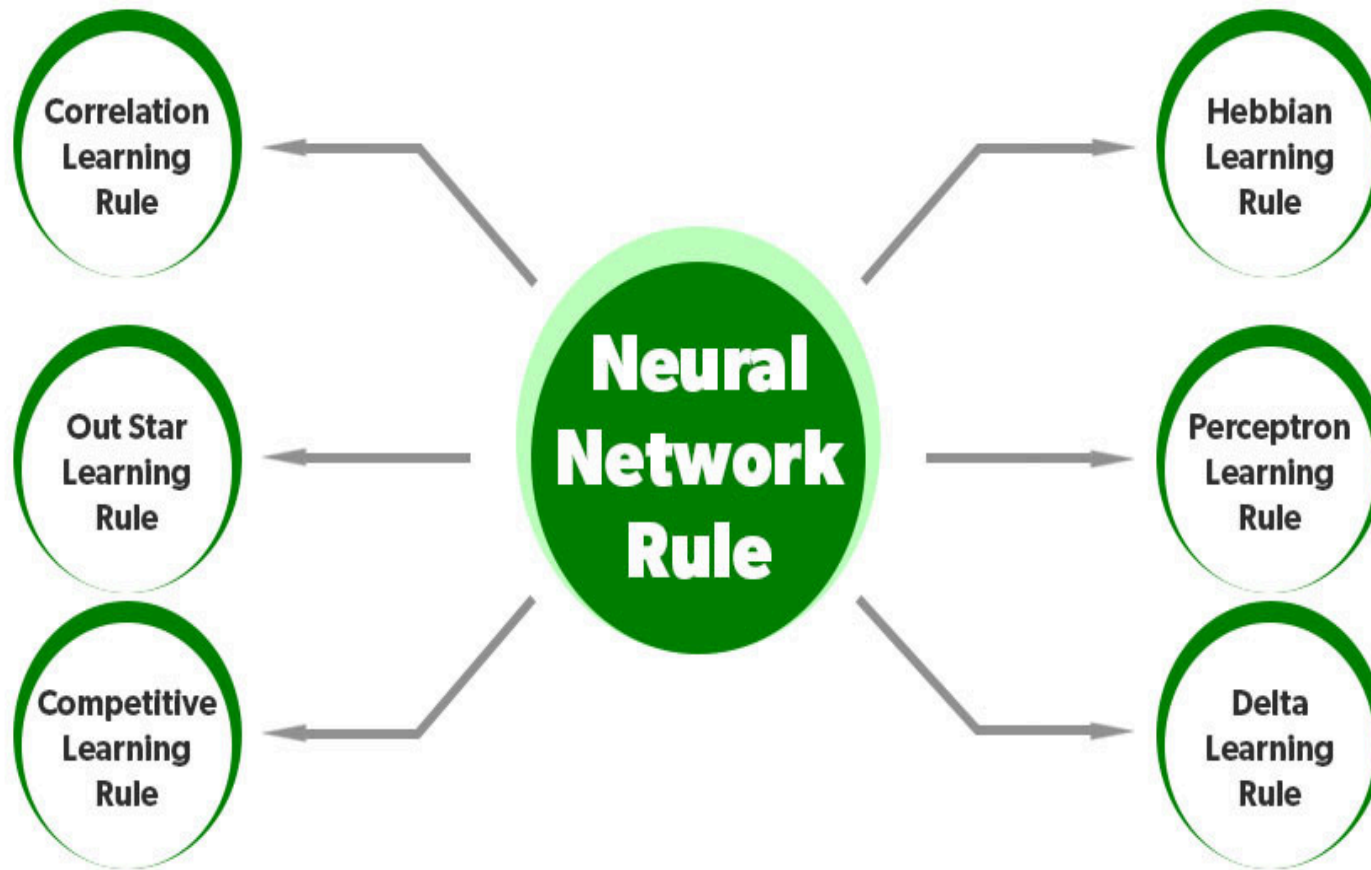


# Learning rules and perceptron learning in deep learning

- Learning rules in artificial neural networks dictate how the network's parameters (weights and biases) are adjusted during the training process to improve its performance.
- These rules aim to minimize the difference between the network's predicted output and the desired target output, a concept known as error correction.
- Different learning rules exist, each with its own mechanism for updating parameters based on the calculated error.



## Hebbian Learning Rule:

A simple rule where weights are increased between neurons that are simultaneously active and decreased between those with opposite activation.

## Perceptron Learning Rule:

Adjusts weights based on the error of individual neurons, suitable for linearly separable data.

## Delta Learning Rule (Error Correction):

Updates weights based on the difference between the predicted and desired outputs, commonly used in backpropagation.

## Correlation Learning Rule:

Updates weights based on the correlation between input and output signals.

## Out Star Learning Rule :

Focuses on updating the weights of connections originating from a specific neuron to multiple target neurons.

## Competitive Learning Rule (Winner-Take-All):

Selects one neuron to be active for a given input, with that neuron's weights being updated.

## Reinforcement Learning Rule:

Principle: Learning occurs by rewarding desirable outcomes and penalizing undesirable ones.

## Boltzmann Learning Rule:

Principle: Inspired by statistical mechanics, this rule uses probabilities to adjust weights.

## Perceptron

**Perceptron** is a type of neural network that performs binary classification that maps input features to an output decision,

usually classifying data into one of two categories, such as 0 or 1.

Perceptron consists of a single layer of input nodes that are fully connected to a layer of output nodes. It is particularly good at learning **linearly separable patterns**. It utilizes a variation of artificial neurons called **Threshold Logic Units (TLU)**, which were first introduced by McCulloch and Walter Pitts in the 1940s. This foundational model has played a crucial role in the development of more advanced neural networks and machine learning algorithms.

## The Perceptron Rule

The Perceptron Rule is an algorithm used to train a perceptron, the simplest type of artificial neural network, designed for binary

classification tasks. It adjusts the weights assigned to the input features based on the errors made in predictions. This allows the perceptron to improve its ability to distinguish between two classes over time, learning the optimal decision boundary between them.

## Perceptron Rule Explained:

### 1. Initialization:

- **Weights and Bias:** Start by initializing the weights (parameters that determine the importance of each input feature) and the bias (a threshold value) to small random values or zeros.
- **Learning Rate ( $\eta$ ):** Choose a learning rate that controls the magnitude of weight updates. A common choice is a small positive value like 0.01.

### 2. Training Process:

- **Input Presentation:** Present each training example to the perceptron one at a time.
- **Weighted Sum Calculation:** For each input vector ( $\mathbf{x}$ ), calculate the weighted sum:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

where ( $\mathbf{w}$ ) is the weight vector,

( $\mathbf{x}$ ) is the input vector,

and ( $b$ ) is the bias.

- **Activation Function:** Apply the activation function (typically a step function) to determine the predicted output  $y_{\text{pred}}$ :

$$y_{\text{pred}} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- **Error Calculation:** Compute the error by comparing the predicted output  $y_{\text{pred}}$  with the actual target  $y$  :

$$\text{Error} = y - y_{\text{pred}}$$

- **Weight and Bias Update:** If there is an error (i.e.,  $y \neq y_{\text{pred}}$ ), update the weights and bias to reduce the error:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \times \text{Error} \times \mathbf{x} \quad b \leftarrow b + \eta \times \text{Error}$$

This adjustment moves the decision boundary in a direction that reduces the classification error for the current example.

### 3. Iteration:

- Repeat the training process for multiple epochs (complete passes through the training dataset) until the perceptron correctly classifies all training examples or until a maximum number of epochs is reached.



## Key Characteristics of the Perceptron Rule:

1. **Linear Separability:** The perceptron rule guarantees convergence only if the two classes are linearly separable, meaning there exists a straight line (in two dimensions) or a hyperplane (in higher dimensions) that can perfectly separate the classes.
2. **Online Learning:** The rule updates weights incrementally for each training example, making it suitable for online learning scenarios where data arrives sequentially.
3. **Simplicity:** Its straightforward update mechanism makes the perceptron rule easy to implement and understand, serving as a building block for more complex neural network training algorithms.

Perceptron Learning Rule is an error-correcting algorithm designed for single-layer feedforward networks. It is a supervised learning approach that adjusts weights based on the error calculated between the desired and actual outputs. Weight adjustments are made only when an error is present.

The process is computed as follows:

## 1. **Definitions:**

- $(x_1, x_2, x_3, \dots, x_n)$ : Set of input vectors
- $(w_1, w_2, w_3, \dots, w_n)$ : Set of weights
- $y$ : Actual output
- $w_0$ : Initial weight
- $w_{\text{new}}$ : New weight
- $\delta w$ : Change in weight
- $\alpha$ : Learning rate

## 2. Computation:

- The actual output is given by:  $y = w_i x_i$
- The learning signal, representing the error, is calculated as:  $e_j = t_i - y$  where  $t_i$  is the desired output.
- The change in weight ( $\delta w$ ) is determined by:  $\delta w = \alpha x_i e_j$
- The new weight is updated as:  $w_{\text{new}} = w_0 + \delta w$

**3. Output Calculation:** The final output is based on the net input and the activation function applied to it.

$$y = \begin{cases} 1, & \text{if net input} \geq \theta \\ 0, & \text{if net input} < \theta \end{cases}$$

This rule provides the foundation for learning in perceptrons, enabling them to make adjustments and improve performance iteratively.

